Comprendre les LLM et maîtriser le prompting

Outils IA · Bachelor 1 · Séance 1 sur 3 · 4 h

Présentation

- Pierre SCHIAVON
- Inetum - Développeur Java Angular
- Ynov Toulouse - Expert en développement Full Stack



ynov CAMPUS

Ce que vous saurez faire à la fin de la séance

Expliquer un LLM

Tokens, mot suivant, contexte, température. Sans mathématiques.

Repérer ses limites

Hallucinations, biais, connaissances datées, réponses qui varient.

Écrire de bons prompts

Rôle, contexte, tâche, contraintes, format, exemples. Puis itérer.

Cadrer un projet avec l'IA

User stories, structure de fichiers, modèle de données.

Le fil rouge des 3 séances : TaskFlow

Une mini-application web de gestion de tâches (HTML, CSS, JavaScript), construite en binôme avec l'aide de l'IA.

Séance 1
Comprendre et cadrer

→

Séance 2
Produire avec l'IA

→

Séance 3
Critiquer et défendre à l'oral

Informatique

Tri, filtres et catégories.

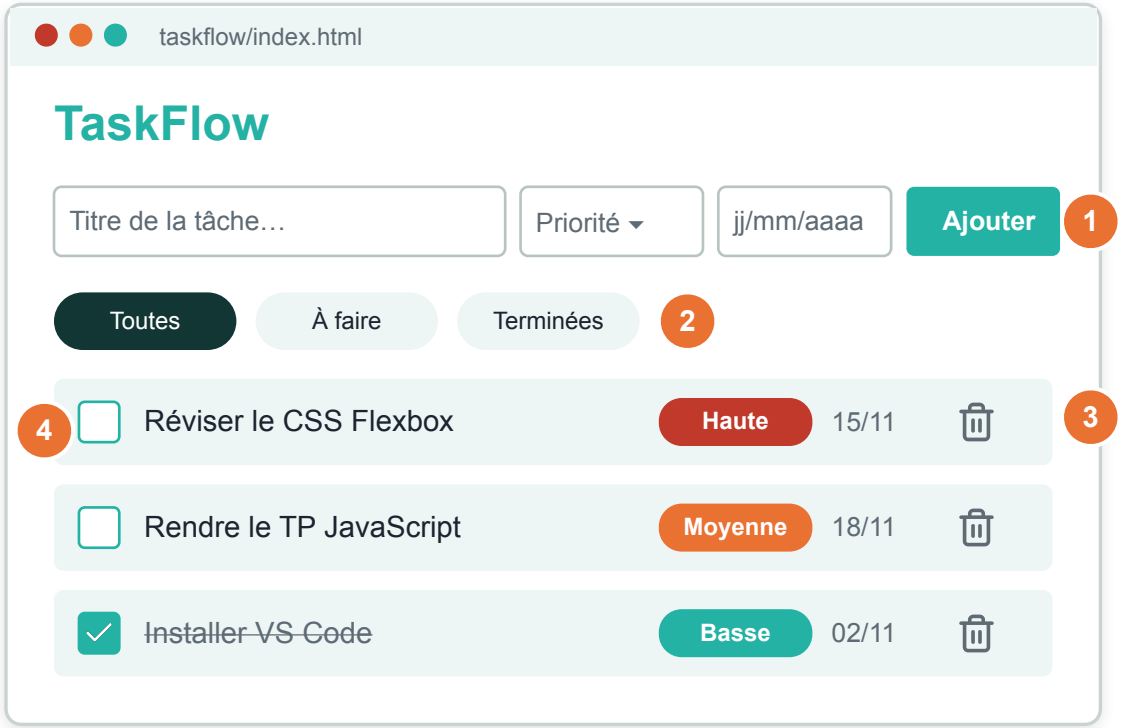
Cybersécurité

Connexion et robustesse des mots de passe.

IA & Data

Statistiques et graphique sur les tâches.

TaskFlow : à quoi ça ressemble



1. **Formulaire d'ajout**
   Titre, priorité et échéance

2. **Filtres**
   Toutes, à faire, terminées

3. **Liste des tâches**
   Priorité en couleur, date d'échéance

4. **Actions**
   Cocher pour terminer, corbeille pour supprimer

Ce que l'application doit savoir faire

Créer

Ajouter une tâche (titre, priorité, échéance). Refuser un titre vide.

Afficher

Voir toutes les tâches, avec leur priorité et leur date.

Modifier

Changer le titre, la priorité ou la date d'une tâche existante.

Terminer

Cocher une tâche faite : elle apparaît barrée.

Supprimer

Retirer une tâche de la liste.

Sauvegarder

Retrouver ses tâches après avoir fermé le navigateur (localStorage).

Votre fonctionnalité filière en détail

Informatique

- Trier par priorité ou par échéance
- Filtrer par statut
- Catégories avec une couleur (cours, perso, projet)
- Bonus : recherche par mot-clé

Cybersécurité

- Écran de connexion avant d'accéder aux tâches
- Contrôle de robustesse du mot de passe : 8 caractères, majuscule, chiffre, caractère spécial
- Jauge faible / moyen / fort
- Bonus : blocage après 3 échecs

Le cadre technique

Ce qu'on utilise

- HTML, CSS et JavaScript, sans framework
- localStorage pour sauvegarder les tâches
- VS Code (ou Cursor) et GitHub
- Séance 1 : Claude, ChatGPT ou équivalent
- Séance 2 : GitHub Copilot et/ou Cursor

Ce qu'on n'utilise pas

- Pas de serveur ni de base de données
- Pas de framework (React, Vue…)
- Pas de bibliothèque externe, sauf pour le graphique en IA & Data
- Pas de données personnelles réelles dans l'application

Ce que vous produisez à chaque séance

Séance	Ce que vous faites sur TaskFlow	Ce que vous rendez
1	Cadrer le projet avec l'IA : user stories, structure de fichiers, modèle de données	README (cahier des charges, user stories) et PROMPTS.md
2	Développer l'application en 3 sprints avec Copilot, Cursor ou Claude	Application fonctionnelle, un commit par sprint, PROMPTS.md à jour
3	Auditer le projet d'un autre groupe, corriger le vôtre, le présenter	Rapport d'audit, version finale du dépôt, oral de 10 min



Un seul dépôt GitHub par binôme, du début à la fin du module.

Déroulé de la séance



Livrables de fin de séance
Un dépôt GitHub par binôme avec un README (cahier des charges, user stories) et un fichier PROMPTS.md avec vos premiers prompts commentés.

1

Comment fonctionne un LLM

Sans mathématiques, promis

Tour de table : et vous, vous en faites quoi ?



- Quels outils d'IA avez-vous déjà utilisés ?
- Pour quoi faire : cours, code, recherche, rédaction, autre ?
- Une fois où l'IA vous a vraiment aidé
- Une fois où elle s'est trompée sans prévenir
- Qu'attendez-vous de ce module ?

Un LLM en trois étapes

1. Entraînement
Il lit d'énormes quantités de textes : web, livres, code

→

2. Modèle
Il en garde des régularités, pas une copie des textes

→

3. Génération
Il prédit la suite d'un texte, morceau par morceau

À retenir : un LLM ne « sait » pas, il prédit ce qui est le plus probable. Une réponse peut donc être fluide, sûre d'elle et fausse.

Les tokens : le texte découpé en morceaux

Le modèle ne lit pas des lettres ni des mots, mais des morceaux de texte appelés tokens.



Exemple de découpage (illustratif) : 1 phrase = 10 tokens. Le « · » représente une espace.

Pourquoi c'est utile

Les limites des outils et leur prix se comptent en tokens, pas en mots.

Conséquence

Compter les lettres d'un mot ou inverser un texte est étonnamment difficile pour un LLM.

Prédire le mot suivant

Le chat dort sur le …

- Le modèle calcule une probabilité pour chaque token possible
- Il en choisit un, l'ajoute au texte, puis recommence
- Une réponse entière = des centaines de prédictions enchaînées

Fenêtre de contexte et température

Fenêtre de contexte

Tout ce que le modèle « voit » au moment de répondre : vos messages, ses réponses, les fichiers fournis.

Instructions | Fichiers | Conversation

Au-delà de la limite, le plus ancien est perdu ou résumé.

Température

Le degré de hasard dans le choix du mot suivant.



Basse
prévisible, répétitive

Haute
créative, moins fiable

D'où des réponses différentes à la même question.

Pourquoi un LLM hallucine

Plausible ≠ vrai

Le modèle produit le texte le plus probable, pas le texte vérifié. Quand il ne sait pas, il ne se tait pas : il invente quelque chose qui ressemble à une bonne réponse.

Exemple : demande « supprime toutes les tâches terminées »

```
// Réponse générée
tasks.removeAll(t => t.done);

// Problème : removeAll()
// n'existe pas en JavaScript

// Version correcte
tasks = tasks.filter(t => !t.done);
```

Les autres limites à connaître

Biais

Il reproduit les biais de ses données : stéréotypes, vision majoritairement anglophone.

Connaissances datées

Il ignore ce qui s'est passé après sa date de fin d'entraînement, sauf s'il peut chercher sur le web.

Réponses variables

Même question, réponse différente. Une réponse isolée ne prouve rien.

Sources inventées

Il peut citer un article, une bibliothèque ou un lien qui n'existe pas.

Quand peut-on faire confiance à la réponse ?

	Erreur sans gravité	Erreur coûteuse
Facile à vérifier	**Utilisez librement** Reformuler, brainstormer, expliquer un concept connu	**Utilisez et vérifiez** Code testable, résumé d'un document que vous avez
Difficile à vérifier	**Avec prudence** Faits précis, chiffres, dates, citations	**À éviter ou faire valider** Sécurité, juridique, santé, données personnelles

Les outils du module

Assistants conversationnels

Claude, ChatGPT, Mistral Le Chat…
Pour réfléchir, expliquer, rédiger, cadrer un projet, déboguer en discutant.

Assistants de code

GitHub Copilot (dans VS Code), Cursor (éditeur complet)…
Pour compléter, modifier et générer du code directement dans le projet.

Séance 1 : assistant conversationnel. Séance 2 : assistants de code.

2

Bien parler à l'IA : le prompting

Démonstration en direct, 45 min

Même demande, deux prompts

Prompt vague

```
Fais-moi une app de tâches.
```

- Langage choisi au hasard
- Fonctions que vous n'avez pas demandées
- Rien sur votre niveau ni vos contraintes

Prompt structuré

```
Tu es un développeur web qui aide
des étudiants de 1re année.
Nous créons TaskFlow, une app de
tâches en HTML, CSS et JavaScript
sans framework. Propose 6 user
stories au format « En tant que…
je veux… afin de… ». Pas de code.
Réponds sous forme de liste.
```

Les 6 briques d'un bon prompt

Rôle

Qui doit répondre : « Tu es un développeur… »

Contexte

Le projet, votre niveau, ce qui existe déjà.

Tâche

Ce que vous voulez, avec un verbe d'action.

Contraintes

Langage, longueur, ce qu'il ne faut pas faire.

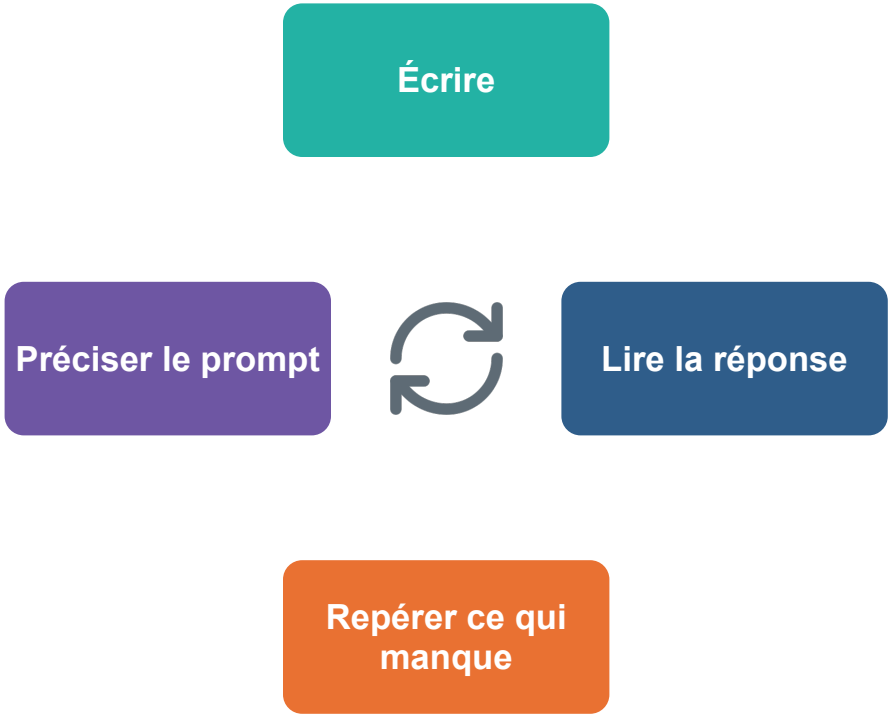
Format

Liste, tableau, code commenté, étapes…

Exemples

Un ou deux exemples de ce que vous attendez.

Le prompt s'améliore par itération



Techniques utiles

- **Étape par étape :** « Explique ton raisonnement avant de répondre »
- **Questions d'abord :** « Pose-moi 3 questions avant de commencer »
- **Hypothèses :** « Liste ce que tu as supposé »
- **Auto-critique :** « Qu'est-ce qui pourrait être faux dans ta réponse ? »

Défi : piégez l'IA



En binôme, 10 minutes : obtenez une réponse fausse mais convaincante.

Pistes

Une citation d'un auteur, la biographie d'un prof, une fonction d'une bibliothèque peu connue, un calcul long.

Puis vérifiez

Comment auriez-vous pu détecter l'erreur ? Documentation, test, autre source ?

3

Exercice : cadrer TaskFlow avec l'IA

En binôme, 2 h

Votre mission en 4 étapes

Binômes
Former le binôme et choisir la fonctionnalité filière · 10 min

→

User stories
Obtenir 5 à 8 user stories en au moins 3 itérations · 40 min

→

Architecture
Structure de fichiers et modèle de données, à critiquer · 40 min

→

Dépôt GitHub
README et PROMPTS.md · 30 min



Règle : l'IA propose, le binôme décide. Gardez une trace de ce que vous avez refusé et pourquoi.

Ce que l'IA doit vous aider à produire

Modèle de données d'une tâche

```
{
  "id": 1,
  "titre": "Réviser le CSS",
  "description": "Flexbox",
  "priorite": "haute",
  "echeance": "2026-11-15",
  "statut": "a_faire"
}
```

Structure de fichiers

```
taskflow/
├── index.html
├── style.css
├── app.js
├── README.md
└── PROMPTS.md
```

Demandez ensuite à l'IA : « Qu'est-ce qui manque à ce modèle pour ma fonctionnalité filière ? »

Le journal de prompts : PROMPTS.md

PROMPTS.md

```
## Prompt 3 · User stories (Claude)

**Prompt :** Tu es un développeur web…

**Résultat :** 8 user stories proposées

**Gardé :** 6 stories, reformulées
**Rejeté :** « synchronisation cloud »
  trop complexe pour notre niveau
**Pourquoi :** on reste sur localStorage
```

Pour chaque prompt important

- Le prompt exact
- L'outil utilisé
- Ce que vous avez gardé
- Ce que vous avez modifié ou rejeté
- Et surtout : pourquoi

Synthèse de la séance

Un LLM prédit

Il ne vérifie pas. Fluide ne veut pas dire juste.

Un prompt se construit

Rôle, contexte, tâche, contraintes, format, exemples.

On itère

Lire, critiquer, préciser. Et garder une trace.

Pour la séance 2 : activez GitHub Copilot (GitHub Student Developer Pack) et/ou installez Cursor. Apportez votre dépôt TaskFlow.